0. **Code:** sotomayorj1594.zip

1. sotomayorj1594 / B.InsertionCounts / Program.cs
2. sotomayorj1594 / C.DictionarySum / Program.cs
3. sotomayorj1594 / TestFinal / Program.cs

1. sotomayorj1594 / B.InsertionCounts / Program.cs

```
namespace B.InsertionCounts
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] array = {
                23,
                1,
                10,
                5,
                2
            };
            Console.WriteLine(string.Join(", ", insertionCount(array)));
        }

        static int[] insertionCount(int[] OrigArray) //1)method accepts array.
        {
            int[] countArray = new int[OrigArray.Length]; //2) return countArray with same length as OrigArray
            int count = 0;

            for (int i = 0; i < OrigArray.Length; i++)
            {
                countArray[i] = OrigArray[i];
            }

            for (int i = 0; i < countArray.Length; i++) //3) nested loop
            {
                for (int j = 1; j ≤ countArray.Length; j++)
```

B1

B1

B2

B2

Not needed

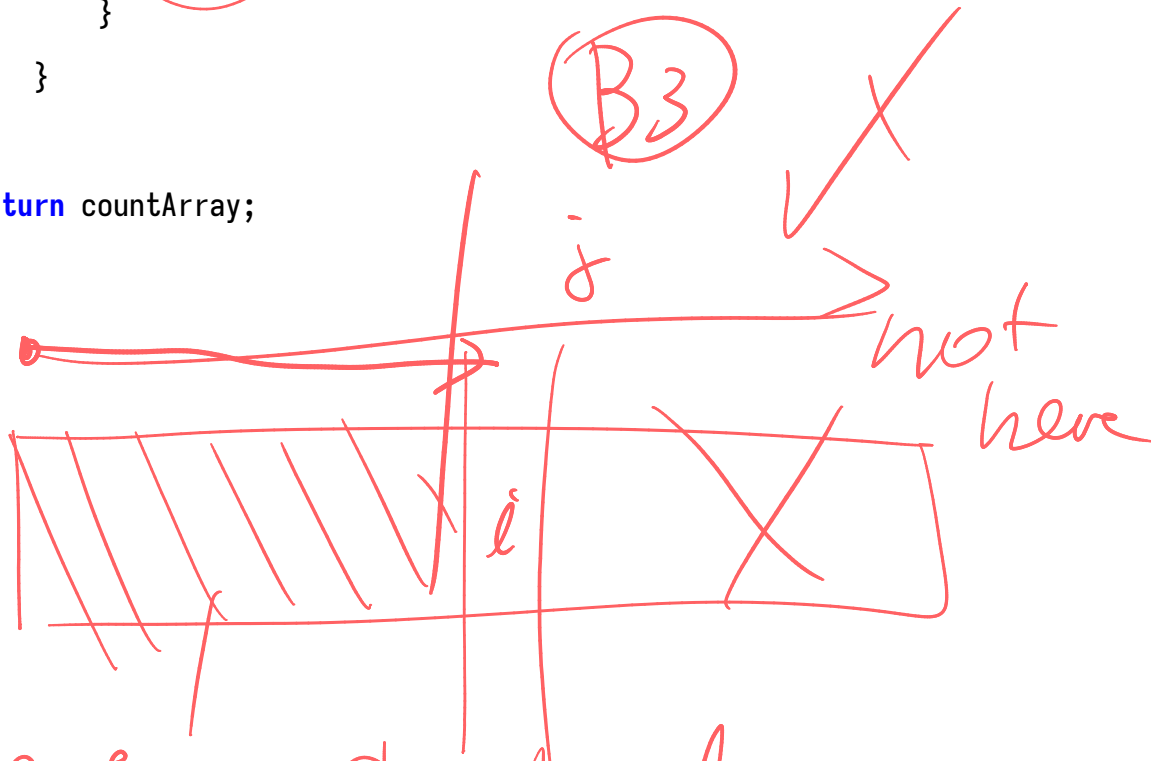
OrigArray

i

1. **sotomayorj1594 / B.InsertionCounts / Program.cs**

```
                {
                    if (countArray[i] > OrigArray[j - 1]) //4) insertstion sort not physically needed.
                    {
                        countArray[i] = count;
                        count++;
                    }
                }
            }
            return countArray;
        }
    }
}
```

B3



Compare here!

Count all of them

B4 Good attempt nonetheless.

2. sotomayorj1594 / C.DictionarySum / Program.cs

```
namespace C.DictionarySum
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] inputA = {
                1,
                2,
                3,
                2,
                7,
                3,
                4,
                4,
                5,
                5,
                4,
                5,
                1,
                6,
                1,
                1
            };
            int result = dictSum(inputA);

            Console.WriteLine("The sum is: " + result);
        }
        static int dictSum(int[] array) //1) method that accepts an array.
        {
```

C1

2. **sotomayorj1594 / C.DictionarySum / Program.cs**

```
31          if (array == null || array.Length == 0) //5) check if array is valid
32          {
33              return 0;
34          }
35          Dictionary < int, int > occDict = new Dictionary < int, int > ();
36          Dictionary < int, int > maxDict = new Dictionary < int, int > (); //6) two dictionaries for O(n)
37
38          foreach(int integer in array) //2) count occurence of each int
39          {
40              if (!occDict.ContainsKey(integer))
41              {
42                  occDict[integer] = 1;
43              }
44              else
45              {
46                  occDict[integer]++;
47              }
48          }
49          int sum = 0;
50          foreach(var kvp in occDict)
51          {
52              int max = 0;
53              if (kvp.Value > max) //3) identify largest number
54              {
55                  max = kvp.Value;
56              }
57              if (!maxDict.ContainsKey(max))
```

C5

C2

num key → count value

max num value → count key

kvp.Value) || maxDict[kvp.Value] < kvp.key)

2. **sotomayorj1594 / C.DictionarySum / Program.cs**

kvp.Value

```
            {
                maxDict[max] = kvp.Key;
                sum = sum + (max * maxDict[max]); //4) sum up the products
            }
        }

        foreach(var kvp in maxDict)
        {
            Console.WriteLine(kvp.Key + " " + kvp.Value);
        }

        return sum;
    }
  }
}
```

separate loop

3. sotomayorj1594 / TestFinal / Program.cs

```
using System.Buffers.Text;

namespace TestFinal
{
    internal class Program
    {
        static void Main(string[] args)
        {
            //1) Enter 2 numbers
            Console.WriteLine("Enter an Interger"); //346251
            int interger = int.Parse(Console.ReadLine());
            Console.WriteLine("Base-N must be positive. Enter base-N digit"); //10
            int baseN = int.Parse(Console.ReadLine());
            int remainder = 0;
            int sum = 0;
            int max = 0;

            while (baseN ≤ 0) //2) Iteratively ensure baseN is positive
            {
                Console.WriteLine("Base-N must be positive. Enter base-N digit");
                baseN = int.Parse(Console.ReadLine());
            }

            if (interger < 0)
            {
                interger = interger * -1;
            }

            while (interger > 0)
```

A1 A5 A2 A3 A3

3. **sotomayorj1594 / TestFinal / Program.cs**

```
            {
                remainder = interger % baseN; //3) Break down intger

                if (remainder > max) //5) determin largest Base-N
                {
                    max = remainder;
                }
                sum += remainder; //4) calculate sum of all baseN digits
                interger = interger / baseN;
            }
            Console.WriteLine($"Sum of base-10 digits = {sum}");
            Console.WriteLine($"Max base-19 digit = {max}");

        }


    }
}
```

A3 A5 A4 A3